Computing for Analytical Work

Session 3 — What broke?

Block 6 — Data files, formats, size, and recovery

This block in one sentence

Data problems often look like code problems. Inspect the input before blaming the code.

The code is the same today as yesterday. The file is not. Look at the file.

Agenda

1. Data files as external reality
2. What a CSV really is: delimiters and header rows
3. Look at the file *before* you load it
4. Five ways a CSV lies, and the `read_csv` option for each
5. The four-line sanity check
6. Git thread — the final clean submission
7. AI thread — hypotheses from the model, validation from the data
8. Lab 6 — The data is the bug, then Homework 3

Data files as external reality

- Your code encodes **assumptions**: this column exists, this is a number, this is a date
- The file was made by someone else, on another machine, with another tool, possibly by hand
- The file does not know about your assumptions and does not care
- Same code, new file, new failure — or worse, new *silent* wrong answer
- Session 1 taught you the filesystem is the source of truth. So is the file's **content**

What a CSV actually is

```
id,name,city
1,Anna,Vienna
2,Bela,Budapest
```

- Plain text; one **row per line**
- A **delimiter** separates the fields — comma by convention, not by law
- The **header row** is the first line, and it is only a header because you say it is
- `pd.read_csv` assumes: comma, header on line 1, `NA`-style spellings for missing

Look at the file before you load it

```
head -5 data/raw/customers.csv
wc -l data/raw/customers.csv
file data/raw/customers.csv
```

- `head -5` — the first five lines: delimiter, header, first values, visible at once
- `wc -l` — how many lines; compare with `df.shape` after loading
- `file` — what kind of file this is; catches a renamed Excel file or a zip pretending to be a CSV
- Thirty seconds, in the terminal, before a single line of Python

Wrong delimiter: what pandas does with it

```
id;name;city
1;Anna;Vienna
2;Bela;Budapest
```

```
df = pd.read_csv("data/raw/customers.csv")
print(df.shape)              # (2, 1)
print(df.columns.tolist())   # ['id;name;city']
```

One column with everything in it. No error. Just a wrong dataframe.

The fix is one argument

```
df = pd.read_csv("data/raw/customers.csv", sep=";")
df = pd.read_csv("data/raw/export.tsv", sep="\t")
print(df.shape)                # (2, 3)
```

- `sep=` tells pandas what the delimiter really is
- You knew which one to pass because you ran `head -5` first
- One column where you expected many is the signature — learn to recognise it

Extra header rows, missing header rows

```
Orders export
Generated 2026-03-01
order_id,customer_id,amount,order_date
1001,7,"€1,200",2026-01-05
```

```
df = pd.read_csv("data/raw/orders.csv", skiprows=2)                # junk above the header
df = pd.read_csv("data/raw/bare.csv", header=None, names=["id", "name", "city"])  # no header at all
```

- Extra lines above: the "header" becomes `Orders export` and every row is misaligned
- No header: your first data row becomes the column names

Missing values, and the many spellings of missing

```
df = pd.read_csv("data/raw/orders.csv", na_values=["-", "n/a", "missing", "?"])
df.isna().sum()
```

- pandas recognises a few spellings by default: empty, `NA`, `NaN`, `null`, `n/a`
- It does **not** recognise `-`, `?`, `missing`, `.`, or `999` — those stay as text
- One `-` in a numeric column turns the **whole column** into strings
- `na_values=` declares your file's spellings; then `isna().sum()` shows you where they were

Numbers stored as text

```
df["amount"].head()              # 0    €1,200   dtype: object
df = pd.read_csv("data/raw/orders.csv", thousands=",")            # handles 1,200
df["amount"] = (df["amount"].str.replace("€", "", regex=False)
                            .str.replace(",", "", regex=False))
df["amount"] = pd.to_numeric(df["amount"], errors="coerce")       # bad ones become NaN
```

- `dtype: object` on a column that should be numeric is the tell
- Strip the symbols, then convert; `errors="coerce"` turns leftovers into `NaN` you can count

Dates as strings

```
df = pd.read_csv("data/raw/orders.csv", parse_dates=["order_date"])
df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce", format="mixed")
df["order_date"].isna().sum()   # how many dates failed to parse?
```

- A date column with `dtype: object` is text, and sorting text dates gives nonsense
- `parse_dates=` for well-behaved files; `to_datetime(errors="coerce")` for messy ones
- Mixed formats (`2026-01-05` , `05/01/2026` , `5 Jan 2026`) need `format="mixed"` and a **count of failures**
- `05/01/2026` is ambiguous — day first or month first? The file's author knows; you must ask

The four-line sanity check

```
df = pd.read_csv("data/raw/sales.csv")
print(df.shape)      # (rows, columns) — is this the size you expected?
print(df.dtypes)     # numbers, not strings? dates, not objects?
df.head()            # what does the data actually look like?
df.isna().sum()      # which columns have gaps, and how many?
```

Run it on every dataframe, every time — as reflexively as you hit save.

What each line catches

- `df.shape` → wrong delimiter (1 column), skipped rows, truncated file
- `df.dtypes` → numbers as text, dates as text, one stray `–` poisoning a column
- `df.head()` → junk header rows, misaligned fields, a currency symbol you did not expect
- `df.isna().sum()` → unrecognised spellings of missing, coercion that silently ate values
- Also look at: `df.columns.tolist()` for names, `df["col"].unique()[:10]` for surprises
- Compare every number to what you expected **before** you loaded the file

Symptom in the code, cause in the file

You see	Look at the file for
`KeyError: 'name'` after load	wrong delimiter, extra header lines, different header spelling
`ValueError: could not convert`	currency symbols, thousands separators, a `-` for missing
`TypeError` on `+` or `sum()`	a text column that should be numeric
A total that is 10× too small	rows dropped by `skiprows`, or coerced to `NaN`
A date sort that looks random	dates still stored as text

Not today: Excel, JSON, encodings, file size

- All four are real and you will meet them this year
- They live on the **Lab 6 reference sheet** — "Other formats and other problems" — in the lab README
- Excel: why opening data in Excel silently changes it
- JSON: what malformed means and how to spot it
- `encoding="latin-1"` when `utf-8` fails on the first byte
- Size: why a file may not open in Excel at all, and what to do instead

Git thread — the final clean submission

`git status` → `git diff` → commit

```
git status
git diff
git add -A
git commit -m "Load orders with correct delimiter, skiprows, and na_values"
git status
```

- `status`: what is modified, what is new, what is untracked junk
- `diff`: read every changed line — is each one something you meant?
- Commit with a message that says **what** and **why**; `status` again should be clean

Before submitting, know exactly what changed

- If `git diff` surprises you, you are not done
- Leftover `print()` calls, a hard-coded absolute path, a commented-out block — these are in the diff, so you will see them
- Generated output and `.venv/` should be ignored, not committed; `git status` shows if they are not
- `git log --oneline` should read like a story of the repair
- Homework 3 asks for this evidence explicitly: status, diff, final commit

AI thread — hypotheses from the model, validation from the data

Three good questions to ask

```
1. Here is the traceback and the first five lines of the file (head -5).
   What are the likely causes of this parsing error?

2. I have a 2 GB CSV I have never seen. How do I inspect it safely
   without opening it in Excel or loading all of it?

3. What does skiprows= do in pd.read_csv, and when would I use
   header= instead?
```

Each one comes back with hypotheses or explanations — **not** with a verified dataframe.

Validation comes from inspecting the data

- The model suggests `sep=";"`. Did `df.shape` change from `(500, 1)` to `(500, 6)`?
- The model suggests `na_values=["–"]`. Does `isna().sum()` show a count you can explain?
- The model suggests `errors="coerce"`. How many values became `NaN`, and which ones?
- **Verification standard:** row counts, column names, dtypes, sample records, the output file exists
- Your AI-use note says what you asked and which of these you checked

5-minute stand-and-stretch

Stand up. Leave the laptop. Back in five for Lab 6.

Lab 6 — The data is the bug

The code in `scripts/load.py` is **mostly reasonable**. The input files are not.

- `customers.csv` — semicolon-delimited
- `orders.csv` — two title lines above the real header
- Amounts like `"€1,200"`
- Dates in **mixed** formats
- Missing values spelled `n/a` and `–`

Start in the terminal, not the editor: `head -5`, `wc -l`, `file`.

Run it, fix the load, validate, note it

```
uv sync
uv run python scripts/load.py
```

- Load each file with the **right options**: `sep=`, `skiprows=`, `na_values=`, `thousands=`, date parsing
- Validate: row count matches `wc -l`, column names match the header, dtypes are right, `isna().sum()` is explainable
- Produce `output/revenue_by_region.csv` and check it exists; one note per data problem in `DIAGNOSIS.md`
- Write the **"what could break next time"** note in `NEXT_TIME.md`: three things, and the check that catches each
- `git status`, `git diff`, commit — then **checkoff**: explain one file's problems and your validation, 60–90 seconds

Homework 3 assigned

Debugging, data files, recovery, and final diagnosis note

- Expected time: **7–8 hours**
- Due: **48 hours before the final exam** — no extensions, the gap is for you to consolidate
- A repo with several realistic failures: **some are code, some are data**
- Repair it, verify the output, explain every failure — and tell code problems from data problems
- Do not expect graded feedback before the exam; review your **own** submission instead

Homework 3 — deliverables

1. Fixed repo with a **clean end-to-end run**
2. **Verified output** file, plus your **data-loading checks**
3. **Final clean commit** — `git status`, `git diff`, commit evidence
4. One **diagnosis note per failure**, standard template
5. **AI-use note** — what you asked, how you verified, if you used it
6. **Video walkthrough, up to 2 minutes** — run it end to end, explain the failures
7. **End-of-course reflection, ~300 words** — next slide

End-of-course reflection — four prompts, ~300 words

1. Of your three diagnosis notes (HW1, HW2, HW3), which is your **strongest**, and why? **Quote one sentence** from it.
2. Where in the loop (observe → locate → hypothesise → inspect → change → verify → explain) did you most often **cut a step**? What did it cost you?
3. Which class of failure was **slowest** for you — paths, environments, notebook state, tracebacks, or data? What is your concrete plan for getting faster next semester?
4. How did your **use of AI change** between HW1 and HW3? One thing you **stopped** doing, one thing you **started**.

300 words is a ceiling. 200 specific words beat 300 general ones.

The final exam

Separate session, closed book, about a week from now. New scenarios, same skills.

The best preparation is re-reading your own three diagnosis notes.